```
class ChainingHashTable:
    """
    Hash Table using Open Hashing (Separate Chaining).
    Each slot stores a chain (Python list) of (key, value) pairs.
    """

    def __init__(self, size):
        """
        Initialize the hash table.
        param size: Table size (m)
        """
        self.size = size
        self.table = [[] for _ in range(size)]

    def _hash(self, key: int) -> int:
        """Hash function: h(k) = k % m"""
        return key % self.size

    def insert(self, key: int, value: int):
        """
        Insert key-value pair using Chaining.
        On collision, append the pair to the end of the chain at that slot.
        """
        if not isinstance(key, int) or not isinstance(value, int):
            print("Error: Only integer keys and values are allowed.")
            return

        # Step 1: Compute the hash value
        # Step 2: Access the chain at that slot
        # Step 3: If the key already exists in the chain, update its value
        # Step 4: Otherwise, append (key, value) to the end of the chain
        chain = self.table[self._hash(key)]
        for index, (stored_key, _) in enumerate(chain):
            if stored_key == key:
                chain[index] = (key, value)
                return
        chain.append((key, value))

    def search(self, key: int):
        """Return the value stored for key, or None if the key is not found."""
        # Step 1: Compute the hash value
        # Step 2: Traverse the chain at that slot and compare each key
        # Step 3: If the chain ends without a match, the key is not in the table
        chain = self.table[self._hash(key)]
        for stored_key, stored_value in chain:
            if stored_key == key:
                return stored_value
        return None

    def display(self):
        """Display the hash table."""
        print("\n--- Final Hash Table (Chaining) ---")
        for i, chain in enumerate(self.table):
            if chain:
                print(f"Slot[{i}] -> " + " -> ".join(str(pair) for pair in chain))
            else:
                print(f"Slot[{i}] -> None")


print("--- Question 1: Chaining ---")
hash_map = ChainingHashTable(size=7)
for k in [10, 17, 24, 31, 38, 45, 52, 59, 66]:
    hash_map.insert(k, k * 10)
hash_map.display()

for key in [24, 45, 70]:
    result = hash_map.search(key)
    if result is not None:
        print(f"Result: Key={key} -> Value={result}")
    else:
        print(f"Result: Key={key} not found.")


# Expected Output:
# --- Question 1: Chaining ---
```

```
# --- Final Hash Table (Chaining) ---
# Slot[0] → None
# Slot[1] → None
# Slot[2] → None
# Slot[3] → (10, 100) → (17, 170) → (24, 240) → (31, 310) → (38, 380) → (45, 450) → (52, 520) → (59, 590) → (66, 660)
# Slot[4] → None
# Slot[5] → None
# Slot[6] → None
# Result: Key=24 → Value=240
# Result: Key=45 → Value=450
# Result: Key=70 not found.


class LinearProbingHashTable:
    """
    Hash Table using Closed Hashing (Linear Probing).
    All (key, value) pairs are stored inside the table itself.
    """

    def __init__(self, size):
        """
        Initialize the hash table.
        param size: Table size (m)
        """
        self.size = size
        self.table = [None for _ in range(size)]

    def _hash(self, key: int) -> int:
        """Hash function: h(k) = k % m"""
        return key % self.size

    def insert(self, key: int, value: int):
        """
        Insert key-value pair using Linear Probing: h_i(k) = (h(k) + i) % m
        """
        if not isinstance(key, int) or not isinstance(value, int):
            print("Error: Only integer keys and values are allowed.")
            return

        # Step 1: Compute the hash value
        # Step 2: If the slot is empty (or holds the same key), store (key, value)
        # Step 3: Collision → probe the next slot (h(k) + i) % m, i = 1, 2, 3, ...
        # Step 4: If every slot was checked, print "Error: Hash table is full."
        start_index = self._hash(key)
        for i in range(self.size):
            index = (start_index + i) % self.size
            entry = self.table[index]

            if entry is None or entry[0] == key:
                self.table[index] = (key, value)
                return

        print("Error: Hash table is full.")

    def search(self, key: int):
        """Return the value stored for key, or None if the key is not found."""
        # Step 1: Compute the hash value and check that slot
        # Step 2: If the slot holds the key, return its value
        # Step 3: If the slot is empty, the key is not in the table
        # Step 4: Otherwise, probe the next slot (wrap around with % m)
        start_index = self._hash(key)
        for i in range(self.size):
            index = (start_index + i) % self.size
            entry = self.table[index]

            if entry is None:
                return None
            if entry[0] == key:
                return entry[1]

        return None

    def display(self):
        """Display the hash table."""
        print("\n--- Final Hash Table (Linear Probing) ---")
        for i, entry in enumerate(self.table):
            print(f"Slot[{i}] -> {entry}")
```

```
print("--- Question 2: Linear Probing ---")
hash_map = LinearProbingHashTable(size=10)
for k in [23, 44, 25, 76, 96, 19, 29]:
    hash_map.insert(k, k * 10)
hash_map.display()

for key in [96, 29, 99]:
    result = hash_map.search(key)
    if result is not None:
        print(f"Result: Key={key} -> Value={result}")
    else:
        print(f"Result: Key={key} not found.")


# Expected Output:
# --- Question 2: Linear Probing ---
# --- Final Hash Table (Linear Probing) ---
# Slot[0] → (29, 290)
# Slot[1] → None
# Slot[2] → None
# Slot[3] → (23, 230)
# Slot[4] → (44, 440)
# Slot[5] → (25, 250)
# Slot[6] → (76, 760)
# Slot[7] → (96, 960)
# Slot[8] → None
# Slot[9] → (19, 190)
# Result: Key=96 → Value=960
# Result: Key=29 → Value=290
# Result: Key=99 not found
```

```
--- Question 1: Chaining ---

--- Final Hash Table (Chaining) ---
Slot[0] -> None
Slot[1] -> None
Slot[2] -> None
Slot[3] -> (10, 100) -> (17, 170) -> (24, 240) -> (31, 310) -> (38, 380) -> (45, 450) -> (52, 520) -> (59, 590) -> (66, 660)
Slot[4] -> None
Slot[5] -> None
Slot[6] -> None
Result: Key=24 -> Value=240
Result: Key=45 -> Value=450
Result: Key=70 not found.
--- Question 2: Linear Probing ---

--- Final Hash Table (Linear Probing) ---
Slot[0] -> (29, 290)
Slot[1] -> None
Slot[2] -> None
Slot[3] -> (23, 230)
Slot[4] -> (44, 440)
Slot[5] -> (25, 250)
Slot[6] -> (76, 760)
Slot[7] -> (96, 960)
Slot[8] -> None
Slot[9] -> (19, 190)
Result: Key=96 -> Value=960
Result: Key=29 -> Value=290
Result: Key=99 not found.
```